

Building Large Language Models

Part 3: Post-Training and Systems

Stanford CS229: Machine Learning

Lecturer: Yann Dubois

August 13th, 2024

Study Notes

Professionally Formatted with TikZ Visualizations

February 1, 2026

Abstract

This document contains comprehensive study notes on **Post-Training and Systems for Large Language Models**, the third part of Stanford's CS229 lecture series on Building LLMs. Part 3 covers the critical phase that transforms base pretrained models into helpful assistants through supervised fine-tuning (SFT) and reinforcement learning from human feedback (RLHF), modern alignment techniques like DPO and PPO, evaluation methodologies for chat models, and the systems optimizations that enable efficient training and inference. The notes include detailed explanations of post-training pipelines, mathematical formulations of preference learning, comprehensive evaluation frameworks including ChatBotArena and AlpacaEval, and systems-level optimizations covering GPU architecture, low-precision training, operator fusion, and parallelization strategies. All visual elements are professionally recreated using TikZ for clarity and long-term reference.

Contents

1	Introduction to Post-Training	4
1.1	Overview	4
1.2	The Post-Training Revolution	4
1.3	Post-Training vs. Pretraining	4
2	Supervised Fine-Tuning (SFT)	6
2.1	Concept and Motivation	6
2.2	SFT Data Creation	6
2.3	SFT Training Details	6
2.4	SFT Limitations	7
3	Reinforcement Learning from Human Feedback (RLHF)	8
3.1	Overview of RLHF	8
3.2	Stage 1: SFT (Covered in §2)	8
3.3	Stage 2: Reward Model Training	8
3.4	Stage 3: RL Optimization with PPO	9
3.5	RLHF Computational Requirements	9
3.6	RLHF Challenges	10
4	Direct Preference Optimization (DPO)	11
4.1	Motivation	11
4.2	DPO Objective	11
4.3	DPO vs. PPO/RLHF	11
4.4	DPO Implementation Details	12
4.5	Visual Comparison: RLHF vs. DPO	13
5	Post-Training Evaluation	14
5.1	Challenges in Evaluating Chat Models	14
5.2	Evaluation Approaches	14
5.2.1	Human Evaluation	14
5.2.2	ChatBotArena / LMSYS Leaderboard	14
5.2.3	AlpacaEval	15
5.2.4	MT-Bench	15
5.2.5	LLM-as-a-Judge	15
5.3	Evaluation Strategy	16
6	Systems and Optimization	17
6.1	Why Systems Matter	17
6.2	GPU Architecture and Fundamentals	17
6.2.1	CPU vs. GPU	17
6.2.2	NVIDIA GPU Generations	17
6.2.3	Tensor Cores	18
6.3	Low-Precision Training	18
6.3.1	Floating Point Formats	18
6.3.2	Mixed-Precision Training	19
6.3.3	Quantization for Inference	19
6.4	Operator Fusion and Kernel Optimization	20
6.4.1	The Memory Bottleneck	20
6.4.2	Operator Fusion	20
6.4.3	FlashAttention	20

6.5	Parallelization Strategies	21
6.5.1	Data Parallelism	21
6.5.2	Model Parallelism (Tensor Parallelism)	21
6.5.3	Pipeline Parallelism	22
6.5.4	Combination: 3D Parallelism	22
6.6	Training Efficiency Metrics	22
6.7	Systems Optimization Visual Summary	23
7	Practical Considerations and Best Practices	24
7.1	Post-Training Pipeline Summary	24
7.2	Data Quality Over Quantity	24
7.3	Cost Comparison	24
7.4	Systems Optimization Priorities	24
7.5	Common Pitfalls	25
8	Conclusion and Future Directions	26
8.1	Key Takeaways	26
8.2	The Complete LLM Pipeline	26
8.3	Future Directions	26
8.3.1	Post-Training	26
8.3.2	Evaluation	27
8.3.3	Systems	27
8.4	Final Thoughts	27
9	Glossary	29
10	References and Further Reading	30
10.1	Key Papers	30
10.2	Online Resources	30
10.3	Course Materials	30

1 Introduction to Post-Training

1.1 Overview

After pretraining on trillions of tokens, language models possess impressive capabilities for next-token prediction but lack the ability to follow instructions, engage in dialogue, or align with human preferences. **Post-training** (also called alignment) is the process of adapting pretrained models to be helpful, harmless, and honest assistants.

Why Post-Training Matters

Pretrained models predict text from the internet distribution—including toxic content, misinformation, and unhelpful responses. Post-training transforms them into:

- **Instruction followers** that understand and execute user requests
- **Conversational agents** that engage in multi-turn dialogue
- **Aligned assistants** that respect human values and preferences

1.2 The Post-Training Revolution

The emergence of ChatGPT in November 2022 demonstrated that post-training techniques could dramatically improve user experience. While GPT-3 was impressive, ChatGPT (based on similar-sized models with RLHF) was transformative.

ChatGPT's Impact

- Reached 100 million users in 2 months (fastest growing consumer app in history)
- Demonstrated that **alignment** > **raw capability** for user satisfaction
- Sparked industry-wide focus on post-training methods
- Same base model + RLHF = Vastly different user experience

1.3 Post-Training vs. Pretraining

Aspect	Pretraining	Post-Training
Objective	Next-token prediction	Follow instructions, align
Data Scale	1-15 trillion tokens	10K-1M examples
Data Quality	Quantity matters most	Quality critical
Cost	\$50-100M	\$100K-1M
Duration	2-3 months	Days to weeks
Compute	10,000s of GPUs	100s of GPUs
Human Labor	Minimal (data filtering)	Extensive (annotation)
Reversibility	N/A (starting point)	Can catastrophically forget

Post-Training is Not Free

While cheaper than pretraining, post-training requires:

- High-quality human annotations (expensive and time-consuming)
- Careful data curation to avoid distribution shift

- Multiple iterations to balance helpfulness vs. safety
- Ongoing evaluation and red-teaming

2 Supervised Fine-Tuning (SFT)

2.1 Concept and Motivation

Supervised Fine-Tuning (SFT) is the first stage of post-training. The goal is to teach the model to follow instructions by fine-tuning on high-quality (instruction, response) pairs.

Supervised Fine-Tuning

Given a dataset $\mathcal{D}_{\text{SFT}} = \{(x_i, y_i)\}_{i=1}^N$ where x_i is an instruction and y_i is a desired response, we fine-tune the pretrained model to minimize the standard cross-entropy loss:

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{SFT}}} \left[\sum_{t=1}^{|y|} \log p_{\theta}(y_t | x, y_{<t}) \right]$$

2.2 SFT Data Creation

Data Quality > Data Quantity

SFT performance is highly sensitive to demonstration quality. A few thousand high-quality examples often outperform millions of lower-quality examples.

Data Collection Approaches:

1. Human-Written Demonstrations

- Professional annotators write responses to instructions
- Highest quality but most expensive (\$5-50 per example)
- InstructGPT: 13K human-written examples

2. Distillation from Stronger Models

- Use GPT-4 or Claude to generate responses
- Much cheaper and scalable
- Alpaca: 52K GPT-3.5 generated examples for \$500
- Risk: Can't exceed teacher model capabilities

3. Hybrid Approaches

- Human-written + AI-generated + human-edited
- Balances cost and quality
- Used by most modern models

2.3 SFT Training Details

Typical Hyperparameters:

- **Learning Rate:** 1e-5 to 5e-5 (much smaller than pretraining)
- **Batch Size:** 32-128 examples
- **Epochs:** 1-3 (overtraining can degrade capabilities)
- **Duration:** Hours to a few days on 8-64 GPUs

Overfitting and Forgetting

SFT can cause:

- **Catastrophic Forgetting:** Lose capabilities from pretraining
- **Distribution Shift:** Overfit to narrow instruction distribution
- **Mitigation:** Early stopping, diverse instruction data, mixing with pretraining data

2.4 SFT Limitations

While SFT enables instruction-following, it has fundamental limitations:

1. **Binary Signal:** No way to express that one response is better than another
2. **Expensive Scaling:** Writing more demonstrations has diminishing returns
3. **Subjectivity:** Single "correct" answer often doesn't exist
4. **Style vs. Substance:** Model may learn formatting without understanding

This motivates **preference learning** via RLHF.

3 Reinforcement Learning from Human Feedback (RLHF)

3.1 Overview of RLHF

RLHF moves beyond demonstrations to learning from human preferences. Instead of "this is the correct answer," humans provide "response A is better than response B."

RLHF Pipeline

The standard RLHF pipeline has three stages:

1. **Supervised Fine-Tuning (SFT):** Train initial policy on demonstrations
2. **Reward Model Training:** Train a model to predict human preferences
3. **RL Optimization:** Optimize the policy to maximize the reward model

3.2 Stage 1: SFT (Covered in §2)

The SFT model becomes the initial policy π_{SFT} for reinforcement learning.

3.3 Stage 2: Reward Model Training

Goal: Train a model to score responses according to human preferences.

Preference Data

Collect comparisons where humans rate responses:

$$\mathcal{D}_{\text{pref}} = \{(x_i, y_i^w, y_i^l)\}_{i=1}^N$$

where x_i is a prompt, y_i^w is the preferred (winning) response, and y_i^l is the dispreferred (losing) response.

Reward Model Architecture:

- Start from SFT model
- Replace final layer with scalar output head
- Train to assign higher scores to preferred responses

Bradley-Terry Model

Assume the probability that y^w is preferred over y^l follows:

$$P(y^w \succ y^l | x) = \frac{\exp(r_\phi(x, y^w))}{\exp(r_\phi(x, y^w)) + \exp(r_\phi(x, y^l))} = \sigma(r_\phi(x, y^w) - r_\phi(x, y^l))$$

where r_ϕ is the reward model and σ is the sigmoid function.

Training Objective:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y^w, y^l) \sim \mathcal{D}_{\text{pref}}} \left[\log \sigma(r_\phi(x, y^w) - r_\phi(x, y^l)) \right]$$

3.4 Stage 3: RL Optimization with PPO

Goal: Optimize the policy π_θ to maximize expected reward while staying close to the initial policy π_{SFT} .

RL Objective

The full objective balances reward maximization with KL divergence penalty:

$$\mathcal{J}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} [r_\phi(x, y) - \beta \cdot D_{\text{KL}}(\pi_\theta(\cdot|x) \parallel \pi_{\text{SFT}}(\cdot|x))]$$

where β controls the strength of the KL penalty.

Why KL Penalty?

- Prevents model from drifting too far from SFT initialization
- Avoids reward hacking (exploiting reward model weaknesses)
- Maintains language modeling capabilities

Proximal Policy Optimization (PPO)

PPO is the standard RL algorithm used for RLHF:

- Updates policy gradually using clipped objective
- Prevents destructively large policy updates
- Requires multiple models in memory (policy, reference policy, reward model, value function)
- Memory intensive: $4\times$ the model size needed during training

3.5 RLHF Computational Requirements

Component	Memory	Purpose
Policy Model π_θ	$1\times$ (trainable)	Generate responses
Reference Model π_{SFT}	$1\times$ (frozen)	KL penalty computation
Reward Model r_ϕ	$1\times$ (frozen)	Score responses
Value Model V_ψ	$1\times$ (trainable)	Estimate value function
Total	$4\times$	

For a 70B parameter model:

- Memory needed: $4 \times 70\text{B} \times 2 \text{ bytes} = 560\text{GB}$
- Requires: $8\times$ A100 80GB GPUs minimum
- Cost: Significantly higher than SFT

3.6 RLHF Challenges

RLHF Difficulties

1. **Instability:** RL training can diverge or collapse
2. **Reward Hacking:** Model exploits reward model weaknesses
3. **Complexity:** Many hyperparameters (learning rates, KL coefficient, etc.)
4. **Compute Cost:** 4× memory requirement limits accessibility
5. **Human Labor:** Collecting preferences is expensive

This motivates simpler alternatives like **Direct Preference Optimization (DPO)**.

4 Direct Preference Optimization (DPO)

4.1 Motivation

DPO simplifies RLHF by **eliminating the RL stage entirely**. Instead of training a reward model and then optimizing with PPO, DPO directly optimizes the policy on preference data.

DPO Key Insight

The optimal policy under the RLHF objective has a closed-form solution:

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{SFT}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

We can rearrange this to express the reward in terms of the policy:

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{SFT}}(y|x)} + \beta \log Z(x)$$

4.2 DPO Objective

Substituting into the Bradley-Terry model and removing the partition function (which cancels), we get the DPO loss:

DPO Loss

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y^w, y^l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y^w|x)}{\pi_{\text{SFT}}(y^w|x)} - \beta \log \frac{\pi_{\theta}(y^l|x)}{\pi_{\text{SFT}}(y^l|x)} \right) \right]$$

Interpretation:

- Increase probability of preferred response y^w
- Decrease probability of dispreferred response y^l
- Stay close to reference policy π_{SFT} (implicit regularization via β)

4.3 DPO vs. PPO/RLHF

Aspect	PPO/RLHF	DPO
Stages	3 (SFT $\rightarrow$ RM $\rightarrow$ RL)	2 (SFT $\rightarrow$ DPO)
Models Needed	4 (policy, ref, reward, value)	2 (policy, reference)
Memory	4 $\times$ model size	2 $\times$ model size
Stability	Lower (RL can diverge)	Higher (supervised learning)
Hyperparameters	Many (LR, KL coeff, clip, etc.)	Few (LR, β)
Training Time	Slower (RL iterations)	Faster (single pass)
Performance	Slightly better (sometimes)	Comparable or better

DPO Advantages

1. **Simplicity:** No reward model, no RL, just supervised learning
2. **Efficiency:** 2 $\times$ memory instead of 4 $\times$
3. **Stability:** No RL instability issues

4. **Speed:** Faster training (no PPO iterations)
5. **Accessibility:** Can run on smaller GPU clusters

4.4 DPO Implementation Details

Typical Hyperparameters:

- **Learning Rate:** $5e-7$ to $5e-6$ (very small)
- β (**KL penalty**): 0.1 to 0.5
- **Batch Size:** 64-256 preference pairs
- **Epochs:** 1-3
- **Duration:** Hours to 1-2 days

DPO Limitations

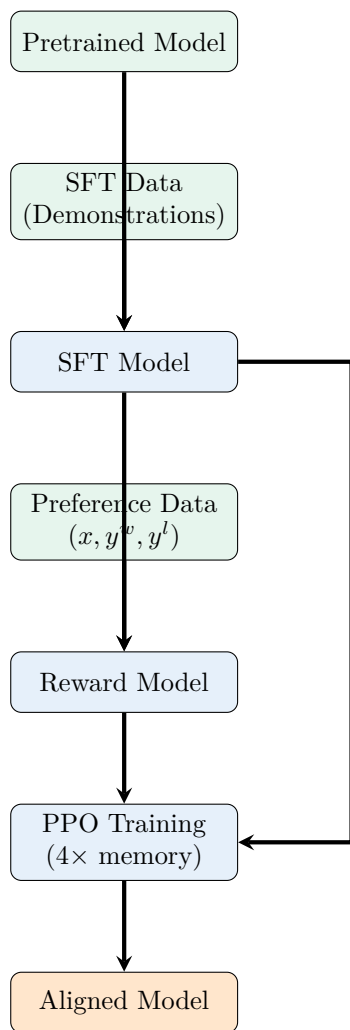
- Requires high-quality preference data (same as RLHF)
- Less flexible than RLHF (can't easily incorporate non-preference rewards)
- May underperform PPO on complex tasks (though rare)

Data Scale Differences

In practice, **preference learning (RLHF/DPO) requires significantly more data than SFT**. While SFT often uses 10K–100K demonstrations, preference tuning commonly uses **100K–1M+ comparison pairs**, because preference signals are weaker and noisier than token-level supervision and require multiple sampled responses per prompt.

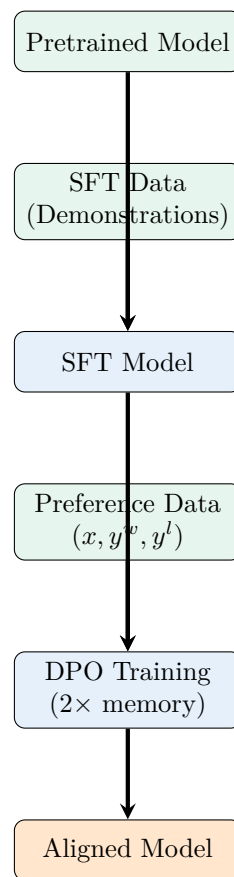
4.5 Visual Comparison: RLHF vs. DPO

RLHF Pipeline



3 stages, complex

DPO Pipeline



2 stages, simple

5 Post-Training Evaluation

5.1 Challenges in Evaluating Chat Models

Evaluating post-trained models is fundamentally different from evaluating pretrained models:

Evaluation Challenges

- **No Single Correct Answer:** Many valid responses to the same prompt
- **Subjectivity:** Quality depends on user preferences
- **Multi-dimensional:** Helpfulness, harmlessness, honesty all matter
- **Context-Dependent:** Same response can be good or bad depending on context
- **Gaming:** Models can learn to exploit evaluation metrics

5.2 Evaluation Approaches

5.2.1 Human Evaluation

Gold Standard but Expensive:

- Have humans rate responses on helpfulness, safety, etc.
- Pairwise comparisons: "Which response is better?"
- Absolute ratings: "Rate this response 1-5 stars"

Challenges:

- Expensive (\$1-10 per comparison)
- Slow (can't iterate quickly)
- Inter-annotator disagreement
- Hard to reproduce

5.2.2 ChatBotArena / LMSYS Leaderboard

ChatBotArena

Open platform where users chat with two anonymous models and vote for which response is better. Uses Elo rating system (like chess) to rank models.

Advantages:

- Real user preferences (not paid annotators)
- Large scale (millions of comparisons)
- Diverse prompts (reflects real usage)
- Continuously updated

How It Works:

1. User submits prompt

2. Two random models (anonymized) generate responses
3. User votes for better response (or tie)
4. Elo ratings updated

LMSYS Leaderboard Rankings (Early 2024)

Top models by Elo score:

1. GPT-4 Turbo (1251 Elo)
2. Claude 3 Opus (1248 Elo)
3. Gemini Ultra (1234 Elo)
4. GPT-4 (1225 Elo)
5. Claude 3 Sonnet (1201 Elo)

Note: Rankings change frequently as new models are added.

5.2.3 AlpacaEval

AlpacaEval

Automated evaluation using a strong LLM (GPT-4) as judge. Models generate responses to 805 test prompts, and GPT-4 compares each response to a reference (text-davinci-003).

Metric: Win rate against reference model (0-100%)

Advantages:

- Fast and cheap (\$10-20 per evaluation)
- Reproducible
- High correlation with human judgments
- Enables rapid iteration

Disadvantages:

- Biased toward GPT-4's preferences
- Can be gamed (verbosity, formatting)
- Not suitable for safety evaluation

5.2.4 MT-Bench

Multi-turn conversation benchmark with 80 high-quality questions across 8 categories:

- Writing, Roleplay, Reasoning, Math, Coding, Extraction, STEM, Humanities

Evaluation: GPT-4 rates responses 1-10

Advantage: Tests multi-turn capabilities, not just single-turn

5.2.5 LLM-as-a-Judge

Using LLMs for Evaluation

Modern trend: Use strong LLMs (GPT-4, Claude) to evaluate other LLMs.

- Much cheaper than human evaluation
- Faster than human evaluation
- Reasonably correlated with human preferences
- Enables continuous evaluation and monitoring

Best Practices:

- Use detailed rubrics
- Multiple judges for important decisions
- Calibrate against human judgments
- Watch for position bias, length bias, self-preference bias

5.3 Evaluation Strategy

Recommended Evaluation Pipeline

1. **Development:** Use automated metrics (AlpacaEval, MT-Bench) for rapid iteration
2. **Pre-Release:** Human evaluation on critical dimensions (safety, bias)
3. **Post-Release:** Monitor ChatBotArena rankings and user feedback
4. **Continuous:** LLM-as-judge for ongoing quality monitoring

6 Systems and Optimization

6.1 Why Systems Matter

While pretraining and post-training focus on *what* to compute, systems engineering focuses on *how* to compute efficiently. Modern LLMs require massive computational resources—systems optimizations can provide:

- **10-100× speedup** through hardware optimization
- **2-4× memory reduction** through quantization
- **5-10× throughput** through batching and parallelization
- **Cost savings** of millions of dollars per training run

Systems Enables Scale

Without systems optimizations, training GPT-4 scale models would be:

- Prohibitively expensive (10× cost)
- Too slow (years instead of months)
- Impossible (exceeds single-GPU memory)

6.2 GPU Architecture and Fundamentals

6.2.1 CPU vs. GPU

Aspect	CPU	GPU
Cores	8-64	1,000s-10,000s
Clock Speed	3-5 GHz	1-2 GHz
Parallelism	Low (task-level)	High (data-level)
Memory Bandwidth	50-100 GB/s	1,000-3,000 GB/s
Best For	Serial tasks	Parallel matrix operations
Deep Learning	Too slow	Essential

Why GPUs for Deep Learning?

- Neural networks = massive matrix multiplications
- Highly parallelizable across examples and parameters
- GPUs have 10-100× higher throughput for these operations

6.2.2 NVIDIA GPU Generations

Generation	Year	Peak FLOPS	Memory	Key Feature
V100	2017	125 TFLOPS (FP16)	32 GB	First Tensor Cores
A100	2020	312 TFLOPS (FP16)	80 GB	2.5× V100
H100	2022	989 TFLOPS (FP16)	80 GB	3× A100
H200	2024	989 TFLOPS (FP16)	141 GB	More memory

H100 (Current Standard):

- 80 GB HBM3 memory (high bandwidth)
- 3 TB/s memory bandwidth
- 989 TFLOPS FP16 (with sparsity)
- Cost: \$25,000-40,000 per GPU

6.2.3 Tensor Cores

Tensor Cores

Specialized hardware units for accelerating matrix multiplication operations. Perform mixed-precision matrix multiply-accumulate:

$$D = A \times B + C$$

where A, B are FP16/BF16 and C, D are FP32.

Performance Impact:

- 8-16× faster than standard CUDA cores for matmul
- Essential for training large models efficiently
- Enabled by modern low-precision training techniques

6.3 Low-Precision Training

6.3.1 Floating Point Formats

Format	Bits	Range	Precision
FP32 (Float)	32	$\pm 10^{38}$	7 decimal digits
FP16 (Half)	16	$\pm 65,504$	3 decimal digits
BF16 (BFloat16)	16	$\pm 10^{38}$	3 decimal digits
INT8	8	-128 to 127	Integer only
FP8	8	Limited	2 decimal digits

BFloat16 vs. FP16

BFloat16 (Brain Float 16):

- Same exponent range as FP32 (8 bits)
- Reduced mantissa (7 bits vs. 23 bits)
- **Better for deep learning:** No overflow issues
- Dominant format for modern training

FP16:

- Larger mantissa (10 bits)
- Smaller exponent range (5 bits)
- Prone to overflow/underflow in training

6.3.2 Mixed-Precision Training

Mixed-Precision Training

Technique that uses lower precision (FP16/BF16) for most operations while maintaining FP32 master weights for numerical stability.

How It Works:

1. Maintain FP32 master weights
2. Convert to BF16 for forward/backward pass
3. Accumulate gradients in FP32
4. Update FP32 master weights
5. Repeat

Benefits:

- **2× memory reduction** (store activations in BF16)
- **2-3× speedup** (Tensor Core acceleration)
- **Same final accuracy** (FP32 master weights prevent precision loss)

Memory Savings Breakdown:

- Model weights: 2× smaller (BF16 vs. FP32)
- Activations: 2× smaller (BF16 vs. FP32)
- Gradients: 2× smaller (BF16 vs. FP32)
- Optimizer states: Still FP32 (no savings here)

6.3.3 Quantization for Inference

Even lower precision for inference (not training):

- **INT8 Quantization:** 4× memory reduction, minimal accuracy loss
- **INT4 Quantization:** 8× memory reduction, small accuracy degradation
- **FP8:** Emerging format (2× reduction, used in H100)

Quantization Impact

For a 70B parameter model:

- FP32: 280 GB
- BF16: 140 GB (training standard)
- INT8: 70 GB (inference)
- INT4: 35 GB (fits on single GPU!)

6.4 Operator Fusion and Kernel Optimization

6.4.1 The Memory Bottleneck

Modern GPUs are often **memory-bound**, not compute-bound:

- Compute: 989 TFLOPS (H100)
- Memory Bandwidth: 3 TB/s
- Loading data from memory takes longer than computation!

6.4.2 Operator Fusion

Operator Fusion

Combining multiple operations into a single kernel to reduce memory transfers.

Example: LayerNorm + Activation
Unfused (Naive):

1. Compute LayerNorm → Write to memory
2. Read from memory → Compute Activation → Write to memory

Total: 4 memory operations

Fused:

1. Read input → Compute LayerNorm → Compute Activation → Write output

Total: 2 memory operations (**2× speedup**)

Common Fusions in Transformers:

- Attention QKV projection fusion
- Softmax + Dropout fusion
- LayerNorm + Residual connection
- Activation + Linear layer

6.4.3 FlashAttention

FlashAttention

Optimized attention implementation that reduces memory reads/writes:

- Standard attention: $O(N^2)$ memory, multiple passes
- FlashAttention: $O(N)$ memory, single fused kernel
- **2-4× speedup** for training
- **10-20× memory reduction** for long sequences
- Enables training on sequences up to 64K tokens

How It Works:

- Tile the attention computation into blocks

- Keep tiles in fast SRAM (on-chip memory)
- Avoid writing intermediate results to slow HBM (GPU memory)
- Fuse softmax, masking, and dropout into single kernel

6.5 Parallelization Strategies

Training large models requires distributing computation across multiple GPUs. Three main strategies:

6.5.1 Data Parallelism

Data Parallelism

Replicate the model on each GPU, split the batch across GPUs, and synchronize gradients.

How It Works:

1. Each GPU has full model copy
2. Each GPU processes different batch subset
3. After backward pass, average gradients across GPUs
4. All GPUs update their model copies identically

Limitations:

- Each GPU must fit entire model (limits model size)
- Communication overhead grows with model size
- Inefficient for models $> 10\text{B}$ parameters

6.5.2 Model Parallelism (Tensor Parallelism)

Tensor Parallelism

Split individual layers across multiple GPUs. Each GPU computes part of each layer.

Example: Split Linear Layer

- Weight matrix $W \in \mathbb{R}^{d \times d}$
- Split into $W_1, W_2, \dots, W_k$ (each $\mathbb{R}^{d \times d/k}$)
- Each GPU computes partial output
- All-reduce to get full output

Advantages:

- Can train models larger than single GPU memory
- Fine-grained parallelism

Disadvantages:

- High communication overhead (all-reduce after each layer)
- Requires fast interconnect (NVLink, InfiniBand)

6.5.3 Pipeline Parallelism

Pipeline Parallelism

Split model into stages (groups of layers), assign each stage to a GPU, and pipeline batches through stages.

How It Works:

1. Split model: Layers 1-10 on GPU 0, 11-20 on GPU 1, etc.
2. Process microbatches in pipeline fashion
3. GPU 0 processes batch 1, passes to GPU 1
4. While GPU 1 processes batch 1, GPU 0 starts batch 2

Challenge: Pipeline Bubbles

- GPUs idle while waiting for previous stages
- Mitigated by using many microbatches

6.5.4 Combination: 3D Parallelism

3D Parallelism

Modern systems combine all three:

- **Data Parallelism:** Across nodes
- **Tensor Parallelism:** Within node (8 GPUs)
- **Pipeline Parallelism:** Across stages

Example: GPT-3 175B training used 3D parallelism on 10,000+ GPUs.

6.6 Training Efficiency Metrics

Model FLOPs Utilization (MFU)

Fraction of theoretical peak FLOPS actually achieved during training:

$$\text{MFU} = \frac{\text{Achieved FLOPS}}{\text{Peak Hardware FLOPS}}$$

Typical MFU Values:

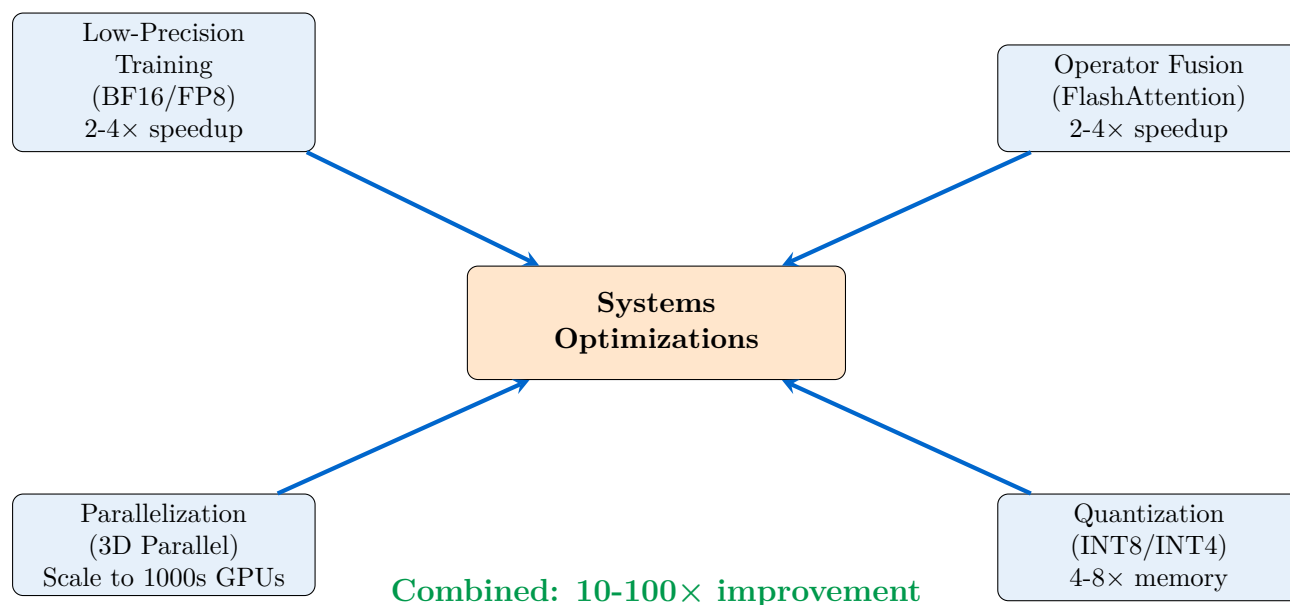
- Naive implementation: 10-20%
- Well-optimized: 40-50%
- State-of-the-art: 55-60%
- Theoretical max: 65% (limited by memory bandwidth)

Impact of MFU

For a training run using 10,000 H100s for 30 days:

- 30% MFU: \$50M in compute costs
- 60% MFU: \$25M in compute costs
- **Doubling MFU = Halving cost!**

6.7 Systems Optimization Visual Summary



7 Practical Considerations and Best Practices

7.1 Post-Training Pipeline Summary

Recommended Post-Training Pipeline

1. **SFT:** Train on 10K-100K high-quality demonstrations
2. **Preference Tuning:** Use DPO (simpler) or PPO (if needed)
3. **Iterative Refinement:** Multiple rounds based on evaluation
4. **Safety Testing:** Red-teaming, adversarial prompts
5. **Deployment:** Monitor real-world performance

7.2 Data Quality Over Quantity

Quality Matters Most

For post-training, **data quality dominates data quantity**:

- 1K excellent examples > 100K mediocre examples
- Spend time curating and filtering
- Use strong models (GPT-4) for data generation
- Have humans review and edit AI-generated data

7.3 Cost Comparison

Stage	Compute	Duration	Human Cost	Total
Pretraining	\$50-100M	2-3 months	\$1-2M	\$51-102M
SFT	\$10-50K	1-3 days	\$50-500K	\$60-550K
RLHF (PPO)	\$50-200K	3-7 days	\$100-500K	\$150-700K
DPO	\$20-100K	1-3 days	\$100-500K	\$120-600K

Key Insight: Post-training is 100-1000× cheaper than pretraining, but human annotation costs dominate.

7.4 Systems Optimization Priorities

For Training:

1. Enable BF16/FP8 mixed precision (easy 2× win)
2. Use FlashAttention (2-4× for attention)
3. Implement gradient checkpointing (reduce memory)
4. Optimize data loading (avoid CPU bottleneck)
5. Profile and fuse operators (case-by-case 10-30%)

For Inference:

1. Quantize to INT8 or INT4 (4-8× memory reduction)
2. Use KV-cache for autoregressive generation
3. Batch requests when possible (higher throughput)
4. Consider distillation to smaller model (10× speedup)

7.5 Common Pitfalls

Avoid These Mistakes

1. **Over-optimizing SFT:** Training too long causes forgetting
2. **Ignoring evaluation:** Can't improve what you don't measure
3. **Chasing benchmarks:** AlpacaEval real-world performance
4. **Reward hacking:** Model exploits reward model weaknesses
5. **Neglecting safety:** Alignment tax is real but necessary
6. **Skipping systems optimization:** Wastes millions in compute

8 Conclusion and Future Directions

8.1 Key Takeaways

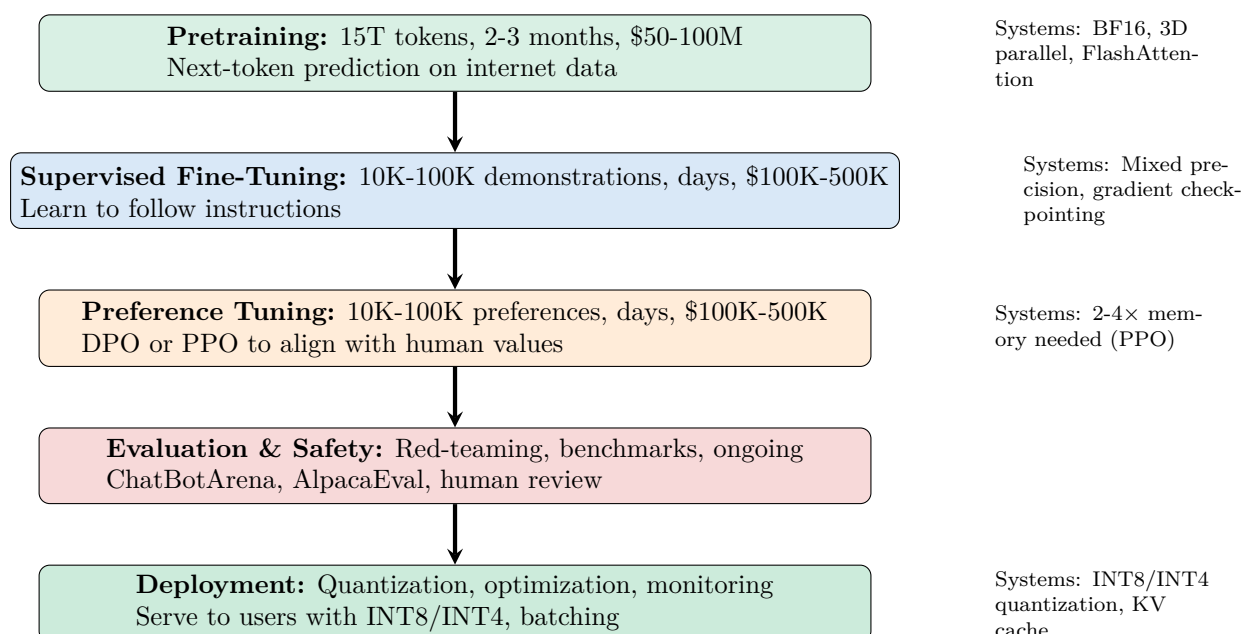
Post-Training Essentials

1. **Post-training transforms capabilities:** Same base model + alignment = ChatGPT
2. **SFT is foundation:** High-quality demonstrations enable instruction-following
3. **DPO > PPO for most:** Simpler, cheaper, more stable
4. **Evaluation is hard:** Use multiple methods (automated + human)
5. **Data quality matters most:** 1K great examples > 100K mediocre

Systems Essentials

1. **Systems enable scale:** 10-100× improvements from optimization
2. **Mixed-precision is essential:** BF16 training is standard
3. **Memory is the bottleneck:** Operator fusion and FlashAttention critical
4. **Parallelization enables large models:** 3D parallelism for 100B+ parameters
5. **MFU matters:** Doubling efficiency halves cost

8.2 The Complete LLM Pipeline



8.3 Future Directions

8.3.1 Post-Training

- **Scalable Oversight:** How to align superhuman AI when humans can't evaluate?

- **Constitutional AI:** Align models to principles, not just preferences
- **Multi-objective Optimization:** Balance helpfulness, safety, truthfulness
- **Personalization:** Different users have different preferences
- **Synthetic Preference Data:** Can we reduce human annotation costs?

8.3.2 Evaluation

- **Better Benchmarks:** Current ones saturating, need harder tasks
- **Long-context Evaluation:** How to evaluate 100K+ token reasoning?
- **Safety Evaluation:** Adversarial robustness, jailbreak resistance
- **Agentic Evaluation:** Testing tool use, planning, multi-step reasoning

8.3.3 Systems

- **FP8 and Below:** Can we train with 8-bit or lower precision?
- **New Hardware:** TPUs, custom AI chips, neuromorphic computing
- **Sparsity:** Leveraging sparse activations and weights
- **Long-context Optimization:** Efficient attention for 1M+ token contexts
- **Inference Optimization:** Speculative decoding, early exit, caching

8.4 Final Thoughts

The field of LLMs has progressed from basic next-token prediction to sophisticated aligned assistants in just a few years. This transformation required advances in:

1. **Pretraining** (Part 1): Data, scale, and architecture
2. **Scaling Laws** (Part 2): Predictable improvement, optimal allocation
3. **Post-Training** (Part 3): Alignment, evaluation, and systems

The Path Forward

Future progress will likely come from:

- Continued scaling (10× per generation for 1-2 more cycles)
- Better post-training methods (beyond RLHF/DPO)
- Improved evaluation (measuring what matters)
- Systems optimization (enabling scale efficiently)
- Novel applications (agents, tools, multimodal)

The techniques covered in these three parts form the foundation for understanding and building modern LLMs. As the field evolves, the fundamental principles—scale, data quality, alignment, and systems optimization—will remain central to progress.

“The bitter lesson is that building systems that leverage computation is ultimately the most effective path forward.”

— Richard Sutton, adapted for the LLM era

9 Glossary

SFT (Supervised Fine-Tuning) Training on (instruction, response) pairs to enable instruction-following

RLHF (Reinforcement Learning from Human Feedback) Three-stage process: SFT → Reward Model → PPO optimization

DPO (Direct Preference Optimization) Simplified alternative to RLHF that skips reward model and RL

PPO (Proximal Policy Optimization) RL algorithm used in RLHF for policy optimization

Reward Model Model trained to score responses according to human preferences

Bradley-Terry Model Statistical model for pairwise preference probabilities

ChatBotArena Platform for crowdsourced LLM evaluation via pairwise comparisons

AlpacaEval Automated evaluation using GPT-4 as judge

LLM-as-a-Judge Using strong LLMs to evaluate other LLMs

Tensor Cores Specialized GPU hardware for accelerating matrix multiplication

Mixed-Precision Training Using BF16 for computation with FP32 master weights

BFloat16 (BF16) 16-bit floating point format optimized for deep learning

Quantization Reducing numerical precision (e.g., FP32 → INT8) for efficiency

Operator Fusion Combining multiple operations into single kernel to reduce memory transfers

FlashAttention Optimized attention implementation with reduced memory usage

Data Parallelism Replicate model, split batch across GPUs

Tensor Parallelism Split individual layers across GPUs

Pipeline Parallelism Split model into stages, pipeline batches through stages

3D Parallelism Combination of data, tensor, and pipeline parallelism

MFU (Model FLOPs Utilization) Percentage of theoretical peak FLOPS achieved

KL Divergence Measure of difference between probability distributions

Catastrophic Forgetting Loss of pretrained capabilities during fine-tuning

Reward Hacking Model exploiting weaknesses in reward model

10 References and Further Reading

10.1 Key Papers

Post-Training and Alignment:

- Ouyang et al. (2022): *Training language models to follow instructions with human feedback* (InstructGPT/ChatGPT)
- Rafailov et al. (2023): *Direct Preference Optimization* (DPO)
- Christiano et al. (2017): *Deep reinforcement learning from human preferences*
- Bai et al. (2022): *Constitutional AI: Harmlessness from AI Feedback*

Evaluation:

- Zheng et al. (2023): *Judging LLM-as-a-judge with MT-Bench and Chatbot Arena*
- Dubois et al. (2023): *AlpacaFarm: A Simulation Framework for Methods that Learn from Human Feedback*

Systems and Optimization:

- Dao et al. (2022): *FlashAttention: Fast and Memory-Efficient Exact Attention*
- Micikevicius et al. (2018): *Mixed Precision Training*
- Shoeybi et al. (2019): *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*
- Rajbhandari et al. (2020): *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*

10.2 Online Resources

- **ChatBotArena Leaderboard:** <https://chat.lmsys.org>
- **AlpacaEval:** https://tatsu-lab.github.io/alpaca_eval/
- **HuggingFace TRL (Transformer Reinforcement Learning):** Tools for RLHF/DPO
- **DeepSpeed:** Microsoft's optimization library
- **NVIDIA Megatron:** Large-scale training framework

10.3 Course Materials

- **Stanford CS229:** Machine Learning (this lecture)
- **Stanford CS324:** Large Language Models
- **Stanford CS336:** Language Modeling from Scratch
- **Stanford CS25:** Transformers United